

Bootstrap Capability: Prompt-Managed Workflow Engineering

A White Paper on Iterative Spec-Driven Development

NOAA EMC Global Workflow MCP Team
Environmental Modeling Center
National Oceanic and Atmospheric Administration
Terry.McGuinness@noaa.gov

January 6, 2026

Abstract

We present **Bootstrap Capability**, an extension to the Spec-Driven Development (SDD) framework that enables AI-assisted workflow engineering through iterative prompt-managed specification development. The system introduces a two-phase execution model: **dry_run** for plan validation and **supervised** for controlled execution with human approval gates. This approach bridges the gap between natural language task descriptions and executable workflow specifications, enabling a rapid development cycle where AI drafts specifications, humans validate through dry-run preview, and execution proceeds with oversight. We demonstrate this capability through implementation in the MCP/RAG system for NOAA Global Workflow, achieving autonomous code generation with full auditability and rollback support.

Keywords: Spec-Driven Development, Bootstrap Capability, Workflow Engineering, AI-Assisted Development, Model Context Protocol

1 Introduction

Modern software development increasingly relies on AI assistance for code generation, documentation, and system maintenance. However, a fundamental challenge remains: how do we ensure AI-generated actions align with human intent while maintaining the efficiency gains of automation?

The **Bootstrap Capability** addresses this challenge by introducing a structured workflow engineering process where:

1. Specifications are written in human-readable Markdown
2. AI assistants can draft these specifications through prompt engineering
3. A **dry_run** mode validates specifications without execution
4. A **supervised** mode executes with human approval at each side-effect step

This paper describes the conceptual foundation, execution model, and practical implementation of this capability within the SDD framework.

2 The Iterative Development Loop

The Bootstrap Capability establishes a feedback loop between prompt engineering (AI-assisted) and system execution, as illustrated in Figure 1.

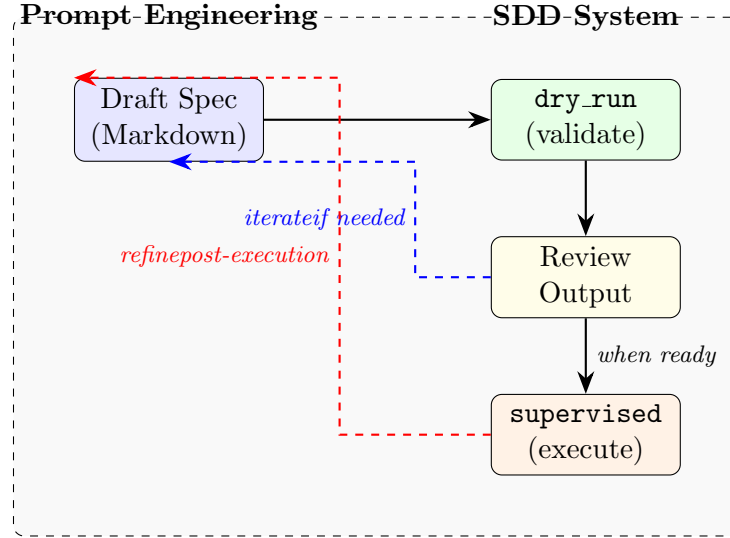


Figure 1: The Iterative Development Loop: AI-assisted specification drafting feeds into the SDD system’s validation and execution phases, with feedback loops for refinement.

2.1 Phase Descriptions

Draft Spec An AI assistant (Claude, Copilot, etc.) or human author creates a workflow specification in Markdown format. The specification defines steps, types, targets, and validation criteria.

dry_run The system parses the specification and reports what actions *would* occur without executing anything. This is analogous to `terraform plan` or Claude CLI’s `/plan` mode.

Review Output The human reviews the dry-run output to verify the plan matches intent. If issues are found, the specification is refined and re-validated.

supervised Once satisfied with the plan, the human triggers supervised execution. The system executes each step, pausing for approval before any side-effect operation (file creation, command execution, database modification).

3 Execution Modes

The Bootstrap Capability distinguishes between two fundamental execution modes, summarized in Table 1.

Table 1: Comparison of Execution Modes

Mode	Creates Code	Runs Commands	Approval Gates	Changes Persist
dry_run	No	No	No	N/A
supervised	Yes	Yes	Yes	Yes

Key Insight

dry_run is **not** sandboxed execution—it is pure specification parsing. No code runs, no files are created, no side effects occur. It answers: “What does this workflow *intend* to do?”

supervised is **real execution** with human oversight. When you approve a step, the action occurs on the live system. It answers: “May I proceed with this action?”

3.1 Side-Effect Classification

The system classifies workflow steps into two categories:

- **Read-only steps** (auto-execute): `health_check`, `validation`, `data_query`, `analysis`
- **Side-effect steps** (require approval): `code_generation`, `code_modification`, `command`, `ingestion`, `file_delete`, `git_operation`

In supervised mode, read-only steps execute automatically while side-effect steps pause for human approval.

4 Workflow Specification Format

Workflows are defined in Markdown with structured metadata. A well-formed step specification includes:

```

1 ### Step 3: Generate Example Tool
2 **Type**: code_generation
3 **Target**: mcp_server_node/src/tools/ExampleTool.js
4 **Template**: basic-mcp-tool
5 **Required**: Yes
6 **Variables**:
7 - className: ExampleTool
8 - toolName: example_operation
9
10 Generate a new MCP tool using the template system.
```

Listing 1: Example Workflow Step Specification

The parser extracts:

- Step name and number (from heading)
- Type (determines execution handler)
- Target (file path, command, etc.)
- Template (for code generation)

- Variables (interpolated into templates)
- Required flag (fail workflow if step fails)

5 Implementation: Bootstrap Capability Demo

We demonstrate the Bootstrap Capability through a workflow that generates a functional MCP tool:

1. **Health Check:** Verify system components are operational
2. **Validate Knowledge Base:** Confirm RAG system is healthy
3. **Generate Code:** Create `ExampleBootstrapTool.js` from template
4. **Validate File Exists:** Confirm file was created
5. **Validate Syntax:** Run `node --check` on generated file
6. **Validate Content:** Verify class definition exists in file
7. **Cleanup** (optional): Remove generated file after demo

5.1 Template System

Code generation uses interpolated templates stored in `src/templates/`:

```

1  /**
2   * {{className}} - {{description}}
3   * Generated by SDD Bootstrap Capability
4   * Date: {{generatedDate}}
5   */
6  export class {{className}} {
7    registerWith(server) {
8      server.tool('{{toolName}}', '{{toolDescription}}', ...);
9    }
10 }
```

Listing 2: Template Variable Interpolation

5.2 Transaction Management

All modifications occur within a transaction that supports rollback:

```

1  BEGIN TRANSACTION 'bootstrap_capability_demo'
2    |-- Create backup directory
3    |-- Execute Step 3 (code_generation)
4    |      +-- Track: file created, path stored
5    |-- Execute Step 4-6 (validation)
6    |      +-- If ANY fails -> ROLLBACK
7    +-- Cleanup (optional)
8  COMMIT TRANSACTION (or ROLLBACK on failure)
```

Listing 3: Transaction Flow

Table 2: Comparison with Related Development Approaches

Approach	Spec Format	Preview Mode	Approval Gates
Traditional Coding	N/A (code is spec)	No	No
Terraform/IaC	HCL/YAML	Yes (<code>plan</code>)	No
Claude CLI	Natural Language	Yes (<code>/plan</code>)	Limited
GitHub Actions	YAML	No (dry-run limited)	Manual triggers
SDD Bootstrap	Markdown	Yes (<code>dry_run</code>)	Yes (per-step)

6 Comparison with Related Approaches

The SDD Bootstrap approach uniquely combines human-readable specifications (Markdown) with both preview and fine-grained approval capabilities.

7 Conclusion

The Bootstrap Capability extends the SDD framework with a practical methodology for AI-assisted workflow engineering:

1. **Specification as Source of Truth:** Markdown workflows define intent before execution
2. **Iterative Refinement:** `dry_run` enables rapid validation without side effects
3. **Controlled Execution:** `supervised` mode provides human oversight at critical points
4. **Auditability:** Transaction management enables rollback and change tracking

This approach bridges the gap between AI capability and human oversight, enabling systems to “bootstrap” their own development while maintaining the safety guarantees required for operational software.

7.1 Future Work

- **Sandboxed Execution:** True isolation via containers or feature branches
- **Multi-Agent Workflows:** USD (Unsupervised Development) for sub-agent task dispatch
- **Automated Spec Generation:** AI generates workflow specs from natural language requirements
- **Continuous Integration:** `auto_approved` mode with manifest-based pre-approval for CI/CD

Acknowledgments

This work was developed as part of the NOAA Global Workflow MCP/RAG project, supporting AI-assisted development for operational weather forecasting systems.